AI PRINCIPLES & TECHNIQUES

PROGRAMMING ASSIGNMENT 2

Radboud University
29 November 2024

Levente Bodi	s1122062
Konstantinos Konstantinou	s1118596

Introduction

Search and optimization algorithms are foundational to artificial intelligence, addressing a wide range of decision-making and problem-solving tasks. Constraint satisfaction problems (CSPs) are a prominent subset of such problems, where the goal is to assign values to variables under strict constraints. One of the most popular CSP examples is Sudoku, a puzzle that requires assigning values to a grid while adhering to specific rules about row, column, and subgrid constraints.

In this report, we focus on the implementation and evaluation of the AC-3 (Arc-Consistency 3) algorithm, a standard technique for ensuring arc consistency in CSPs. The AC-3 algorithm iteratively eliminates inconsistent values from variable domains, simplifying the problem and reducing the search space. To further enhance its efficiency, we experiment with various heuristics which influence the order of arc processing, potentially improving runtime and reducing redundant evaluations.

The task at hand was to implement the AC-3 algorithm, represent Sudoku puzzles as CSPs, and integrate heuristic optimizations. We conducted experiments across multiple Sudoku puzzles of varying difficulty to compare performance metrics. This report documents the implementation process, experimental setup, results, and an analysis of the impact of heuristics on algorithmic efficiency.

Method

Constraint Satisfaction Problems (CSPs)

Constraint Satisfaction Problems involve finding assignments for a set of variables that satisfy a set of constraints. CSPs are defined by three core components:

1. **Variables (V)**:
   The variables are the entities that require value assignments.
2. **Domains (D)**:
   Each variable is associated with a domain, which is the set of possible values that the variable can take.

3. **Constraints (C)**:
   Constraints define relationships between variables that must hold true for the solution to be valid.

The goal of solving a CSP is to assign values to all variables such that all constraints are satisfied. Solutions are typically found by employing constraint propagation techniques to reduce the size of domains, combined with search algorithms for exploring potential assignments.

CSPs are often solved by enforcing local consistency (e.g., arc consistency), which simplifies the problem by systematically eliminating inconsistent values from domains. Algorithms like the aforementioned AC-3 are used to enforce arc consistency by iteratively revising variable domains to ensure compatibility with constraints.

Representing Sudoku as CSP

In the context of Sudoku, the problem is represented as a CSP as follows:

1. **Variables (Xij)**:
   Each cell in the 9x9 grid is treated as a variable, where *i* and *j* represent the row and column indices. This results in 81 variables.
2. **Domains (D(Xij))**:
   The domain of each variable consists of the numbers {1,2,3,4,5,6,7,8,9}. For pre-filled cells, the domain is restricted to the given value. For empty cells, the domain initially contains all values from 1 to 9.
3. **Constraints (C)**:
   Constraints ensure that:
   - Each row contains unique values.
   - Each column contains unique values.
   - Each 3x3 subgrid contains unique values.

These constraints are encoded as arcs between variables. An arc (*Xi*, *Xj*) represents a binary relationship between two variables, indicating that their values must satisfy the corresponding constraint. For example, two cells in the same row are connected by an arc because their values must be distinct.
By representing Sudoku as a CSP, the problem is converted into a structured format that allows constraint-solving algorithms like AC-3 to prune invalid assignments efficiently.

Arc Consistency 3 Algorithm (AC-3)

AC-3 is an algorithm designed to enforce arc consistency in CSPs. A variable Xi is arc-consistent with Xj if every value in the domain of Xi has a corresponding value in the domain of Xj that satisfies the constraint between them. If no such value exists, inconsistent values are removed from D(Xi).
AC-3 iteratively processes arcs in the CSP until no further changes occur or a domain becomes empty (indicating that no solution exists). By reducing domains early in the solving process, AC-3 simplifies the problem and reduces the need for exhaustive search.
Key Steps in AC-3:

1. Initialization: All arcs in the CSP are added to a queue.
2. Revision: For each arc (Xi, Xj) in the queue, the algorithm checks if all values in D(Xi) are supported by at least one value in D(Xj). If not, the inconsistent values are removed from D(Xi).
3. Propagation: If the domain of Xi changes, all arcs involving Xi and its neighbors are re-added to the queue for further consistency checks.
4. Termination: The algorithm terminates when the queue is empty (all arcs are consistent) or when any domain becomes empty (indicating unsolvability).

```
function AC-3(CSP) returns boolean
        // Ensures arc consistency for a given CSP
        persistent: queue // List of arcs, initially containing all arcs

        while queue is not empty do
        (Xm, Xn) ← REMOVE-FIRST(queue)
        REVISE(CSP, Xm, Xn)

        if new size of Dm = 0 then
                return false // No solution exists

        if Dm has been changed then
                for each Xk in {neighbors of Xm} - Xn do
                        if (Xk, Xm) not on queue then
                                add (Xk, Xm) to queue

        return true // CSP is arc-consistent
```

AC - 3 Pseudocode

AC-3 in the Context of Sudoku

In this assignment, AC-3 was implemented to solve Sudoku puzzles by enforcing arc consistency across all constraints (rows, columns, and subgrids). The implementation process was as follows:

1. **Queue Initialization**:
    - Instead of creating arcs for all pairs of variables sharing constraints, arcs are only created for variables that are not yet finalized. This is because finalized variables do not require further arc consistency checks; their values are fixed and cannot change.
    - For each constraint (row, column, or subgrid), arcs are added between a variable Xi and every other variable Xj within the same constraint group, provided Xi has more than one possible value in its domain.
2. **Revision**:
    - For each arc, the REVISE function iterates over the domain of Xi and checks if each value has at least one supporting value in the domain of Xj.
    - If a value in D(Xi) is found to be inconsistent, it is removed.

3. **Propagation**:
    - Whenever a domain is revised, all neighbors of the affected variable are added back to the queue to ensure consistency is maintained.
4. **Termination**:
    - The algorithm terminates when the queue is empty, signifying that arc consistency has been achieved. If any variable's domain becomes empty, the Sudoku is declared unsolvable.

Heuristics for Optimization

To experiment with the performance of AC-3 in this assignment, several heuristics were implemented to prioritize arc processing:

1. **Default:** No heuristic; processes arcs in the order they are added to the queue.
2. **Minimum Remaining Values (MRV):** Prioritizes variables with smaller domains first.
3. **Degree Heuristic:** Focuses on variables involved in the most constraints.
4. **Priority to Finalized Variables:** Targets arcs (Xi, Xj) where Xj has a single value domain.
5. **Combined Heuristics:** All combination of the above heuristics.

The use of heuristics aimed to reduce redundant computations by focusing on arcs most likely to impact the solution, thus improving runtime efficiency.

Time complexity

To evaluate the practical efficiency and complexity of the implemented AC-3 algorithm and its heuristic optimizations, two key metrics were considered:

1. **Number of Arcs Evaluated:**
   This metric reflects the computational workload of the algorithm. AC-3 iteratively processes arcs to enforce consistency, and the number of arcs evaluated directly measures the effort required. By comparing this metric across different heuristics, we can gain insight into how efficiently the algorithm reduces domains and propagates constraints.
2. **Runtime:**
   Runtime measures how long the algorithm takes to complete. While it gives a practical sense of the algorithm's speed, runtime can vary due to factors like system performance. Comparing runtime alongside the number of arcs evaluated helps us understand how efficiently the algorithm and its heuristics perform in practice.

By analyzing these metrics together, we can assess the trade-offs between computational effort and time efficiency for the different heuristics applied. This provides a clear picture of how effectively AC-3 solves Sudoku puzzles.

Results

All five Sudoku puzzles used were solvable, however, among the five Sudoku puzzles used:

- **Sudokus 1, 2, and 5** were ***solvable*** *with AC-3*, meaning arc consistency alone was sufficient to find a solution.
- **Sudokus 3 and 4** were ***unsolvable*** *with AC-3*, as the algorithm terminated due to lack of arcs in the queue, highlighting cases where AC-3 is insufficient without additional backtracking or search strategies.

The results of the heuristic exerimentations are presented in the following tables:

Sudoku 1

Heuristic	Arcs evaluated	Runtime (s)
NONE	1939	0.04479
MRV	1611	0.04083
Degree Heuristic	2262	0.0745
Priority to Finalized	1000	0.07559
MRV & Degree Heuristic	2222	0.0901
MRV & Priority to Finalized	1000	0.07787
Degree Heuristic & Priority to Finalized	1000	0.10693
ALL	1000	0.12271

Sudoku 2

Heuristic	Arcs evaluated	Runtime (s)
NONE	1996	0.04638
MRV	2264	0.05415
Degree Heuristic	2117	0.08014
Priority to Finalized	1020	0.07841
MRV & Degree Heuristic	2144	0.0974
MRV & Priority to Finalized	1020	0.08661
Degree Heuristic & Priority to Finalized	1020	0.11703
ALL	1020	0.1309

Sudoku 3

Heuristic	Arcs evaluated	Runtime (s)
NONE	1894	0.04902
MRV	2665	0.05414
Degree Heuristic	2049	0.07931
Priority to Finalized	1040	0.07851
MRV & Degree Heuristic	2030	0.09603
MRV & Priority to Finalized	1040	0.0913
Degree Heuristic & Priority to Finalized	1040	0.10924
ALL	1040	0.12811

Sudoku 4

Heuristic	Arcs evaluated	Runtime (s)
NONE	1307	0.03899
MRV	2313	0.04343
Degree Heuristic	1358	0.06478
Priority to Finalized	940	0.06047
MRV & Degree Heuristic	1359	0.07871
MRV & Priority to Finalized	940	0.04687
Degree Heuristic & Priority to Finalized	940	0.08553
ALL	940	0.08032

Sudoku 5

Heuristic	Arcs evaluated	Runtime (s)
NONE	1795	0.03987
MRV	2432	0.04368
Degree Heuristic	2028	0.06592
Priority to Finalized	920	0.06721
MRV & Degree Heuristic	2012	0.08013
MRV & Priority to Finalized	920	0.07705
Degree Heuristic & Priority to Finalized	920	0.09504
ALL	920	0.11025

Analysis of Results

The results reveal distinct patterns in the performance of the AC-3 algorithm with different heuristics. The **default heuristic** processes arcs in the order they are added, performing well on simpler puzzles but struggling with more complex ones due to its lack of prioritization. The **MRV heuristic** reduces arcs by focusing on variables with the smallest domains, performing well on simpler puzzles like Sudoku 1 but suffering from sorting overhead in harder puzzles. Similarly, **Priority to Finalized Variables** consistently reduced the number of arcs across all puzzles but incurred slightly higher runtime due to queue updates. **Combined heuristics** which included Priority to Finalized Variables all had the same number of arcs as Priority to Finalized Variables, however, adding more heuristics, as in "ALL," often increased runtime without significant gains in performance.

In conclusion, the AC-3 algorithm has a worst-case time complexity of $O(n*log(n))$ where n is the total number of arcs. While all heuristics share this theoretical complexity, their practical performance varies. Heuristics which included the Priotity to Finalized Variables heuristic perforemed consistently and best in terms of number of arcs evaluated. On the contrary, perhaps due to the constant re-ordering of the queue based on the heuristic, simpler heuristics performed better according to runtime.

Discussion

The experiments underscore the influence of heuristic strategies on the practical performance of the AC-3 algorithm:

1. **Impact of Heuristics**:
    - *Simpler Puzzles*: MRV excels at reducing arcs in simpler puzzles like Sudoku 1 and 5 but suffers from increased runtime overhead on harder puzzles.
    - *Priority to Finalized Variables*: This heuristic is consistently effective in reducing arcs across all puzzles but at the cost of slightly increased runtime.
    - *Combined Heuristics*: Balanced combinations like "MRV & Priority to Finalized" perform well overall but require careful consideration to avoid excessive overhead, as seen in the "ALL" heuristic.
2. **Solvable vs. Unsolvable Cases**:
   While AC-3 effectively solved Sudokus 1, 2, and 5, its inability to handle Sudokus 3 and 4 highlights the algorithm's limitations. In these cases, backtracking or other search strategies would be needed to resolve inconsistencies and find solutions.
3. **Trade-offs Between Arcs and Runtime**:
   The results emphasize the trade-off between the number of arcs evaluated and runtime. For example, heuristics like "MRV" reduce arcs but can increase runtime due to prioritization overhead. Conversely, "Priority to Finalized Variables" minimizes arcs but takes longer to process due to frequent queue updates.

Conclusion

The findings demonstrate that while AC-3 is a robust algorithm for enforcing arc consistency, its practical performance heavily depends on the chosen heuristic. Tailoring heuristics to puzzle complexity offers the best balance between computational workload and runtime efficiency.

Acknowledgments

We would like to express our gratitude to our teacher and the TAs for their guidance throughout this project. Additionally, we would like to thank Juan Navarro and Luca Laber from Group 44 for their assistance and collaborative input during the project.